8. Technical Debt Identification and Management

> Required by the examination paper, Part A, section 7. Every item below records **Debt → Cause → Impact → Priority → Proposed Resolution**, and is classified as *acceptable temporarily*, *scheduled for future resolution*, or *critical and requiring immediate attention*.

8.1 Definition and framing

The metaphor originates with Cunningham [21]; the four-way classification used throughout this section is Fowler's quadrant [24].

Session 3 [3] defines technical debt as *the long-term cost incurred when developers take shortcuts or implement suboptimal solutions in order to achieve short-term goals such as faster delivery*, and **self-admitted technical debt (SATD)** as debt the developer explicitly documents in code comments.

This project treats that literally. Debt here is not something discovered by inspecting the finished system: it was **identified before it was incurred**, and every item is marked in the source at the site that carries it.

8.2 How debt entered this project: three distinct routes

Route	When	Items	Character
The scope decision	Phase 1, before any code	TD-01… TD-06	Chosen. The effort estimate exceeded the budget by 23%, so 4.1 hours of implementation quality were removed deliberately (Section 4.6).
Design decisions	Phase 2, before any code	TD-07… TD-13	Consequential. The layered architecture, the free-tier constraint and deferred requirements each carry a known cost.
Implementation and testing	Phase 3–4	TD-14… TD-17	Discovered. Found while building and while writing tests, the only items not anticipated.
Change request CR-002	Phase 6	TD-18	Consequential. Reinstating FR-31 (SMS) carried a known cost: free-tier hosting provides no worker, so there is no queue.

Thirteen of eighteen items were identified **before the code that carries them existed**. That is the substantive claim of this section, and it is why the debt register and the effort estimate are the same document read twice.

8.3 Self-admitted technical debt in the source

Session 3's SATD markers, present in the shipped code:

```
app/domain/rules.py:41              TODO(TD-13)   cycle length is a constant
app/domain/rules.py:174             FIXME(TD-11)  summary recomputed per render
app/infrastructure/db.py:28         FIXME(TD-01)  create_all, no migrations
app/infrastructure/models.py:231    HACK(TD-09)   audit append-only by convention
app/infrastructure/repositories.py:6 TODO(TD-07)  hand-written mapping
app/services/collection.py:344      FIXME(TD-16)  N+1 on the route sheet
app/services/collection.py:338      TODO(TD-05)   static route sheet
app/services/security.py:38         FIXME(TD-14)  no login rate limiting
app/web/auth.py:52                  TODO(TD-15)   no password reset
app/web/client.py:60                FIXME(TD-16)  N+1 in cycle history
app/web/supervisor.py:65            TODO(TD-04)   bare reversal form
app/web/templates/base.html:10      TD-02         Tailwind via CDN
requirements.txt:5                  TODO(TD-12)   unpinned dependencies
```

Each marker names the register ID, so a developer reading the code reaches the analysis, and a reader of this document reaches the code. Markers are used with Session 3's conventional force: `TODO` for work not done, `FIXME` for something that works but is wrong, `HACK` for a knowingly poor mechanism.

8.4 The debt register

Critical: requiring immediate attention

TD-14 · No login rate limiting or account lockout `app/services/security.py:38` · Code · Prudent & Deliberate

Debt	Authentication accepts unlimited attempts. No backoff, no lockout, no CAPTCHA, no per-IP throttle.
Cause	Rate limiting needs either a shared store (Redis) or a database-backed attempt counter. Neither fitted the 48-hour budget, and the free tier provides no Redis.
Impact	An online brute-force attack against a savings system is possible. Argon2id makes each guess expensive, which raises the cost of an attack but does not bound it. Failed attempts are audited, so an attack is *visible afterwards*; nothing *prevents* one. Phone numbers are the username, and Ghanaian mobile numbers are highly guessable in format.
Priority	**Critical.** The single most serious item in this register.
Resolution	Add an `login_attempts` table keyed on phone and IP; exponential backoff after 3 failures; lock for 15 minutes after 10. Estimated 2–3 hours. Must ship before any real client data enters the system.

TD-15 · The collector sets the client's first password — ✅ **REPAID** `app/services/passwords.py` · Code · Prudent & Inadvertent

Debt	At enrolment the collector typed the client's initial password. There was no forced change at first login and no reset flow at all.
Why it was Critical	It attacked the system's own premise. SusuBook exists so the client holds a record independent of the collector (BR-02). A collector who knows the client's password can sign in as them, making that independence nominal.
Resolution as implemented	**Both halves closed.** A `must_change_password` flag is set on every account created at enrolment, and an application-wide guard redirects the user to the change page and permits nothing else until they replace it, so the collector's password stops working before any record is displayed. Self-service reset by SMS one-time code was added alongside: six digits, stored only as an Argon2 hash, ten-minute expiry, single use, five verification attempts, three requests per hour.
What unblocked it	The debt register named the absent SMS gateway as the cause. **CR-002 delivered that gateway**, and the blocker ceased to exist, so the register was stale within the same working session. That is the staleness Session 3 warns about, caught by re-reading the register rather than by anything systematic.
Residual	The reset path depends on SMS, which is itself allowlist-restricted (CR-002), so a real client outside the allowlist still cannot self-serve. That limit belongs to FR-31's rollout, not to this item.
Verified by	30 unit tests, 13 system tests (TC-PWD)

TD-09 · Audit log is append-only by convention, not by enforcement

`app/infrastructure/models.py:231` · Architecture/Design · Prudent & Deliberate

Debt	Nothing in the application updates or deletes `audit_log` rows, but the database does not prevent it. The application role holds full UPDATE and DELETE.
Cause	Enforcement needs a separate least-privilege database role, or a rule/trigger rejecting UPDATE and DELETE. Both need migration infrastructure that TD-01 removed.
Impact	The audit trail carries non-repudiation (BR-05, NFR-09), it is the evidence that answers a dispute between client and collector. A compromised application account, or an operator with the production connection string, could rewrite history and leave no trace. The guarantee is currently a promise about the code rather than a property of the system.
Priority	**Critical.** The whole value of an audit trail is that it cannot be edited.
Resolution	Revoke UPDATE and DELETE on `audit_log` from the application role; grant INSERT and SELECT only. Add a `BEFORE UPDATE OR DELETE` trigger raising an exception as defence in depth. Estimated 1–2 hours once migrations exist, so it is sequenced after TD-01.

Scheduled for future resolution

TD-01 · Schema created with `create_all()`; no versioned migrations `app/infrastructure/db.py:28` · Infrastructure · Prudent & Deliberate

Debt	`Base.metadata.create_all()` plus a seed script. No Alembic, no migration history, no downgrade path.
Cause	Cut deliberately in the scope decision to recover 0.7 hours (Section 4.6).
Impact	Any schema change in production requires hand-written DDL against live data, with no review artefact and no rollback. Blocks TD-09's resolution. Risk grows the moment real data exists, because `create_all` cannot alter an existing table.
Priority	**Scheduled: first item.** It is a prerequisite for several others.
Interest paid, observed	Repaying TD-15 required a new column on a populated table. `create_all()` cannot alter an existing table, so the change had to be hand-written as idempotent DDL in `MANUAL_MIGRATIONS` with a bespoke `flask db-upgrade` command and a Cloud Run job to run it. That file and that command exist **only** because this debt is unpaid; with Alembic neither would. This is the first time the cost was actually incurred rather than predicted.
Resolution	Introduce Alembic; autogenerate an initial revision matching the current schema; make it a deployment step. Estimated 2 hours.

TD-16 · N+1 queries on the route sheet and cycle history `app/services/collection.py:344`, `app/web/client.py:60` · Code · Prudent & Inadvertent

Debt	`route_sheet` queries the active cycle once per client; `history` runs two queries per cycle.
Cause	Not anticipated, found while auditing the code during Phase 4. The list comprehension reads cleanly and hides the repeated call. Genuinely inadvertent.
Impact	The route sheet is the collector's most-loaded screen, opened dozens of times a day on mobile data, and it is the screen NFR-01's 2-second budget most applies to. Cost grows linearly with route size: fine at 20 clients, poor at 200.
Priority	**Scheduled.** Not yet user-visible at demonstration scale, and it becomes so predictably.
Resolution	Replace with a single join returning clients and their active cycles together; add a repository method rather than looping in the service. Estimated 1–2 hours.

TD-02 · Tailwind loaded from CDN, unpurged `app/web/templates/base.html:10` · Infrastructure · Prudent & Deliberate

Debt	The full Tailwind runtime is fetched from a CDN and compiles classes in the browser. No build step, no purge, no self-hosting.
Cause	Cut in the scope decision to recover 1.0 hour.
Impact	A render-blocking third-party request on every page load, working directly against NFR-01 (2 s on a 3G connection) and CO-07 (low-end phones on mobile data), the exact users this system targets. Also a third-party availability dependency and a supply-chain surface.
Priority	**Scheduled.**
Resolution	Add a Tailwind build producing a purged stylesheet served from the application. Estimated 1–2 hours.

TD-12 · Dependencies are version ranges with no lockfile `requirements.txt:5` · Infrastructure · Prudent & Deliberate

Debt	`requirements.txt` uses `>=` bounds. No lockfile, no hash pinning.
Cause	Ranges avoided install failures during a time-boxed build.
Impact	Builds are not reproducible: the deployed application may not match what was tested, and a transitive release can break production without any change to this repository. Also weakens the reproducibility the examiner would need to rebuild the submission.
Priority	**Scheduled.**
Resolution	Pin exact versions and commit a lockfile (`pip-compile` or `uv lock`). Estimated 30 minutes.

TD-10 · Cycle maturity is evaluated on read, not by a scheduler `app/infrastructure/repositories.py` (`list_due_for_payout`) · Architecture/Design · Prudent & Deliberate

Debt	A cycle becomes visible as matured when a supervisor opens the payouts screen, not at midnight on its end date. No background job exists.
Cause	Free-tier hosting provides no scheduler or worker process (CO-04).
Impact	If no supervisor looks, nothing happens: no notification, no automatic transition. A client whose cycle matured is not paid until someone opens a screen. Correctness is preserved, the query is date-driven, but timeliness depends on human attention.
Priority	**Scheduled.**
Resolution	Add a scheduled task marking cycles MATURED and notifying supervisors, once hosting supports a worker. Estimated 2 hours plus hosting cost.

TD-18 · SMS is fire-and-forget: no delivery receipt, no retry, no queue
`app/services/notifications.py` · Architecture/Design · Prudent & Deliberate

Debt	A message is dispatched on a daemon thread and the outcome is logged, not stored. There is no delivery receipt, no retry on failure, and no queue.
Cause	A durable queue needs a worker process, which free-tier hosting does not provide, the same constraint as **TD-10**. Dispatching inline was the only option that kept the send off the collector's critical path.
Impact	If the gateway is down or the message is rejected, the client is never told and no one knows. The audit log records that a send was *dispatched*, not that it was *delivered*, so the record cannot answer "I was never notified". The contribution itself is unaffected, the ledger is committed before any send is attempted.
Priority	**Scheduled.** The system is correct without it; the notification is an assurance layer, and its silent failure degrades that assurance rather than the record.
Resolution	Persist an outbox row per message, dispatch from a worker, record the provider's delivery status against it, and retry with backoff. Estimated 3–4 hours, and dependent on hosting that provides a worker (as TD-10 also is).

TD-17 · No structured logging, error tracking or monitoring Application-wide · Process · Prudent & Deliberate

Debt	Default Flask logging to stdout. No structured logs, no error aggregation, no uptime monitoring, no alerting.
Cause	Not required to demonstrate the lifecycle, and out of the time budget.
Impact	A production failure is discovered by a user reporting it. Diagnosis depends on whatever the platform retained. Directly limits the *corrective maintenance* capability described at Section 11.1 (The four maintenance categories, applied).
Priority	**Scheduled.**
Resolution	Structured JSON logging with a request id; error tracking (Sentry free tier); uptime check against a health endpoint. Estimated 2–3 hours.

Acceptable temporarily

TD-07 · Hand-written mapping between domain entities and ORM models
`app/infrastructure/repositories.py:6` · Architecture/Design · Prudent & Deliberate

Debt	Seven `_to_*` functions convert records to entities by hand. A schema change must be made in two places.
Cause	The direct consequence of keeping the domain layer framework-free (NFR-07). SQLAlchemy imperative mapping onto the dataclasses would remove it but was not affordable.
Impact	Duplication, and silent drift if a column is added to a model but not its mapper. Partially mitigated: an integration test asserts money round-trips the mapping exactly.
Priority	**Acceptable.** This is the price of an architecture that pays for itself, it is what makes 156 unit tests run in 0.36 s with no database. Removing the mapping would remove that benefit.
Resolution	Revisit only if entity count grows substantially. Consider SQLAlchemy imperative mapping. Extend round-trip tests to every field in the meantime.

TD-08 · Sinkhole: simple reads pass through the service layer `app/services/` · Architecture/Design · Prudent & Deliberate

Debt	Session 5's named anti-pattern: reads such as listing clients traverse the service layer adding little transformation.
Cause	Accepted at design time (Section 7.6).
Impact	A few extra function calls; negligible at this scale.
Priority	**Acceptable.** The cost buys one consistent path for authorisation and audit. Bypassing the layer for reads would create two paths and invite an unauthorised read.
Resolution	None planned. This is a deliberate trade, documented so it is not mistaken for an oversight.

TD-11 · Cycle summaries recomputed on every render `app/domain/rules.py:174` · Code · Prudent & Inadvertent

Debt	`compute_cycle_summary` sums the full contribution list each time a card is displayed; no cached or denormalised total.
Cause	Recognised while designing the card view rather than beforehand — Fowler's "we now know how we should have done it".
Impact	O(n) where a running total would be O(1). Bounded at 31 rows per cycle, so it is not a present problem.
Priority	**Acceptable.**
Resolution	Denormalise a running total onto the cycle row if cycle length ever becomes configurable beyond a month.

TD-03 · Client list has no search, filter or pagination · `app/web/collector.py` · Code · Prudent & Deliberate Cut for 0.5 h. FR-08 is therefore only **partially** satisfied and is reported as such at Section 9.7 (Requirements verification). Degrades past roughly 50 clients; QR scanning (FR-40) bypasses it for the common case. **Acceptable.** Resolution: server-side filter plus pagination, ~1 h.

TD-04 · Reversal is a bare reference-and-reason form · `app/web/supervisor.py:65` · Code · Prudent & Deliberate Cut for 0.6 h. The supervisor must copy a reference by hand rather than acting from the contribution row, which invites transcription error though a mistyped reference fails safe, since no contribution matches. **Acceptable.** Resolution: reverse action on each row, reason codes, ~1 h.

TD-05 · Route sheet is a flat list · `app/services/collection.py:338` · Code · Prudent & Deliberate Cut for 0.5 h; unordered by geography and unfiltered by outstanding status. FR-23 met literally, collector efficiency reduced. **Partially mitigated by CR-001**, scanning bypasses the list for the common case, which is why the reduced version was acceptable. **Acceptable.** Resolution: filter to uncollected, order by route, ~1 h.

TD-06 · HTMX applied to three interactions only · `app/web/` · Architecture/Design · Prudent & Deliberate Cut for 0.8 h. Partial updates on the route sheet's collect action; full page loads elsewhere. Inconsistent interaction model. **Acceptable**, the three chosen sit on the collector's critical path, so the benefit is concentrated where it matters. Resolution: extend to remaining forms, ~1 h.

TD-13 · Cycle length and commission policy are constants · `app/domain/rules.py:41` · Code · Prudent & Deliberate FR-36 (configurable institutional defaults) was deferred as *Could*. Cycles snapshot their own length and rate at open, so making these configurable later cannot disturb existing cycles, and `CommissionPolicy` is already a Strategy interface. **Acceptable**, the design has been left open (Open/Closed) even though the feature is absent. Resolution: settings table plus admin UI, ~2 h.

8.5 Classification summary

By urgency (as the examination requires)

Classification	Count	Items
Critical: immediate attention	2	TD-09, TD-14
Repaid	1	~~TD-15~~ — closed during Phase 6, both halves
Scheduled for future resolution	7	TD-01, TD-02, TD-10, TD-12, TD-16, TD-17, TD-18
Acceptable temporarily	8	TD-03, TD-04, TD-05, TD-06, TD-07, TD-08, TD-11, TD-13
Total	**18**	of which 1 repaid

> **All three critical items were security items, and none was a deliberate scope cut.** One (TD-15) has since been repaid: see below.

> **On TD-15's repayment.** Its recorded cause was the absent SMS gateway. CR-002 delivered that gateway, which meant a **Critical** item silently became unblocked and the register did not notice. It was caught by re-reading the register, not by any process. That is a small, concrete instance of Lehman's second law: a register left alone drifts out of step with the system it describes, and keeping it current is work that has to be done deliberately.

> **The original observation still stands for the rest.** TD-09 follows from a scope cut (no migrations), TD-14 from a platform constraint, TD-15 from an excluded requirement. The time-boxed trade-offs that *were* chosen deliberately all landed in the acceptable band. The time-boxed trade-offs taken knowingly therefore all fell within the acceptable band, while the items posing the greatest risk arose without a decision being made. This supports a general observation: deliberate debt is more readily managed than debt incurred unintentionally, because a stated cause carries a stated resolution with it.

By Fowler's quadrant (Session 3)

	Deliberate	Inadvertent
Prudent	**15 items** — TD-01…10, 12, 13, 14, 17, 18. Taken knowingly, for a stated reason, with an intended repayment.	**3 items** — TD-11, TD-15, TD-16. "We now know how we should have done it."
Reckless	**0**	**0**

No item is reckless. Every one has a recorded cause and a resolution, which is the distinction Session 3 draws between managed and unmanaged debt.

By type (Session 3's six categories)

Type	Count	Items
Code debt	8	TD-03, 04, 05, 11, 13, 14, 15, 16
Architecture / design debt	6	TD-06, 07, 08, 09, 10, 18
Infrastructure debt	3	TD-01, 02, 12
Process debt	1	TD-17
Test debt	**0**	—
Documentation debt	**0**	—

Zero test debt and zero documentation debt is a deliberate result, not an oversight. Session 3 identifies both as arising "when teams skip testing to meet deadlines" and "when teams focus only on coding" — the two most common casualties of a time-boxed build, and the two this project protected. The delivered system carries 209 tests at 97% coverage, and the documentation was written before the code rather than after it. Quality was reduced in styling, in convenience features and in operational tooling instead.

8.6 Interest analysis

Session 3 describes debt accruing *interest*: increased bug frequency, higher maintenance cost, reduced development speed. Estimating where interest is already accruing:

Item	Interest being paid now	Rate
TD-01 no migrations	Blocks TD-09's fix; every future schema change costs manual DDL	**Compounding**, it gates other repayments
TD-14 no rate limiting	None yet; the entire cost arrives at once if exploited	**Step function**
TD-15 collector knows password	Accruing silently, every day the claim of client independence is weaker than stated	**Compounding**
TD-16 N+1	Proportional to route size; invisible at demo scale	**Linear**
TD-02 CDN Tailwind	Paid on every page load by every user, today	**Constant, ongoing**
TD-07 mapping	Paid once per schema change	**Per-event**
TD-08, TD-11, TD-13	Effectively zero at current scale	**Negligible**

TD-01 is sequenced first not because it is the most dangerous but because it compounds and gates. TD-09 cannot be repaid without it.

8.7 Repayment sequence

Order	Item	Estimate	Rationale
1	TD-01 migrations	2 h	Prerequisite for TD-09; unblocks all future schema work
2	TD-14 rate limiting	2–3 h	Highest severity; independent of everything else
3	TD-15 forced password change	3–4 h	Restores the system's central guarantee
4	TD-09 audit log enforcement	1–2 h	Needs TD-01
5	TD-12 dependency pinning	0.5 h	Cheap; makes the rest reproducible
6	TD-16 N+1	1–2 h	Before client counts grow
7	TD-02 Tailwind build	1–2 h	Directly serves NFR-01
8	TD-17 logging and monitoring	2–3 h	Enables corrective maintenance
9	TD-03…06 convenience features	~4 h	User-facing polish, no correctness risk
	Total	**~19–24 h**	Roughly one working week

Items 1–4 (**8–11 hours**) must complete before the system handles real client money. This sequence is carried into the repayment plan in Section 11 (Maintenance and Future Evolution).

8.8 Debt deliberately *not* taken on

Recorded because refusing debt is as much a decision as accepting it, and these were live temptations under time pressure:

Shortcut available	Would have saved	Why it was refused
Floats for money	~0.5 h	BR-R1. A rounding error in a savings ledger is a correctness *and* trust failure, and it is close to unfixable once data exists.
Business rules on the ORM models	~1 h	Would have required a database for every rule test, and those tests would then not have been written at all.
Skipping database-level invariants	~1 h	The domain checks already pass; the indexes exist for the case where the domain layer is wrong.
Editing contributions in place instead of reversals	~0.6 h	Would destroy the non-repudiation that answers a client-collector dispute, the reason the system exists.
Hiding actions in templates instead of enforcing authorisation server-side	~1 h	Hiding a link is presentation, not a control.
Skipping the audit log	~1 h	BR-05. Without it there is no evidence trail and the system solves nothing.

Roughly five hours of additional shortcuts were available and refused. Each would have created debt in the one area, correctness of the client's money and the record of it, where this system cannot afford any.